

I. 动态词嵌入 (22 points)

1. 什么是动态词嵌入? (2 points) 相比静态词嵌入, 使用动态词嵌入有什么优势? (4 points)
2. 在拥有相同大小词表的情况下, 分析比较静态词嵌入和动态词嵌入在使用时的计算效率. (3 points)
3. CoVe 和 ELMo 分别使用什么预训练任务来训练动态词嵌入? (3 points) 他们选择这些预训练任务的动机分别是什么? (4 points)
4. ELMo 在预训练时使用交叉熵 (Cross Entropy) 作为损失函数. 请分别给出 ELMo 在训练动态词嵌入时所使用的正向语言模型和后向语言模型的损失函数公式. (6 points)

[提示 1] 交叉熵损失是一种常用的训练分类模型的损失函数. 它的定义为

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^N y_i \log p_i. \quad (1)$$

其中, N 为类别数; y_i 为第 i 个类别的真实标签 ($y_i = 1$ 表示样本属于该类别, 否则 $y_i = 0$); $p_i \in [0, 1]$ 为模型预测的该样本属于第 i 个类别的概率, 并满足 $\sum_{i=1}^N p_i = 1$.

[提示 2] 对于一个长度为 L 的句子, ELMo 使用的正向语言模型和后向语言模型分别表示为

$$\vec{P}(w_1, w_2, \dots, w_L) = \prod_{t=1}^L P(w_t | \mathbf{x}_{1:t-1}; \vec{\theta}_{\text{LSTM}}, \theta_{\text{out}}), \quad (2)$$

$$\overleftarrow{P}(w_1, w_2, \dots, w_L) = \prod_{t=1}^L P(w_t | \mathbf{x}_{t+1:n}; \overleftarrow{\theta}_{\text{LSTM}}, \theta_{\text{out}}). \quad (3)$$

假设词表大小为 N , 第 t 个单词 w_t 可以看作词表中的某一个类别.

解:

1. **动态词嵌入**是指根据上下文动态调整词向量表示的模型 (如 ELMo、BERT 等), 能够捕捉词语在不同语境下的语义变化。相比静态词嵌入 (如 Word2Vec、GloVe), 其优势包括:
 - **处理一词多义**: 动态词嵌入能根据上下文生成不同表示, 解决静态嵌入中词义固定的问题。
 - **捕捉复杂语义关系**: 通过双向上下文建模, 更准确地反映语法和语义依赖。
 - **任务适应性**: 生成的词向量可直接适配下游任务, 减少特征工程的依赖。
2. **计算效率比较**:
 - **静态词嵌入**: 推理时仅需查表操作, 时间复杂度为 $O(1)$, 计算效率极高。
 - **动态词嵌入**: 需通过神经网络 (如 LSTM、Transformer) 生成上下文相关表示, 时间复杂度为 $O(L \cdot d^2)$ (L 为序列长度, d 为隐藏层维度), 计算开销显著增加。
3. **CoVe 与 ELMo 的预训练任务及动机**:

- **CoVe**: 使用机器翻译 (MT) 作为预训练任务。动机是通过翻译任务中的跨语言对齐, 学习上下文敏感的语义表示。
- **ELMo**: 使用双向语言模型 (前向 + 后向 LM)。动机是结合双向上下文信息, 克服单向语言模型仅能捕捉单侧上下文的局限性。

4. ELMo 的损失函数公式:

- 前向语言模型损失:

$$\mathcal{L}_{\text{forward}} = - \sum_{t=1}^L \log P(w_t \mid \mathbf{x}_{1:t-1}; \vec{\theta}_{\text{LSTM}}, \theta_{\text{out}})$$

- 后向语言模型损失:

$$\mathcal{L}_{\text{backward}} = - \sum_{t=1}^L \log P(w_t \mid \mathbf{x}_{t+1:L}; \overleftarrow{\theta}_{\text{LSTM}}, \theta_{\text{out}})$$

II. 预训练微调范式 (30 points)

1. 什么是预训练微调范式? (2 points) 预训练微调范式相比“从零开始训练 (training from scratch)”的优势在哪里? (4 points)
2. 在预训练阶段, 通常使用自监督学习范式训练模型。什么是自监督学习? (2 points) 使用自监督学习进行预训练的优势是什么? (2 points)
3. 微调阶段有两种常见策略: (1) 冻结预训练参数只训练输出头; (2) 不冻结预训练参数而训练整个模型 (全量微调)。分析并判断哪一种策略可以防止在少样本微调下的过拟合问题。 (3 points) 分析并判断哪一种策略的训练成本更高。 (3 points)
4. 请基于 BERT 预训练模型 bert-mini-uncased 在 Yelp Reviews 数据集上微调文本分类模型。相关的代码, 数据集和参考代码可从以下链接下载。如果受限于设备, 可以使用使用较小规模的数据集进行训练或测试。

<https://box.nju.edu.cn/d/186680aa6273470399db/>

给出的参考代码实现了全量微调, 请运行代码并给出微调过程中各个 epoch 在测试集上的实验结果。 (2 points) 尝试修改代码实现冻结预训练参数的微调, 请解释修改的代码, 并与全量微调的实验结果做比较。 (6 points) 尝试修改代码实现“从零开始训练 (training from scratch)”, 请解释修改的代码, 并与全量微调的实验结果做比较。 (6 points)

解:

预训练微调范式 (30 points)

1. 预训练微调范式包含两个阶段:

- **预训练阶段**：在大规模通用数据上训练模型，学习通用特征表示。
- **微调阶段**：在特定下游任务的小规模数据上进一步调整模型参数，适配具体任务。

优势（相比从零开始训练）：

- **数据高效性**：利用预训练模型学到的通用特征，减少对标注数据量的依赖。
 - **计算高效性**：避免从零训练，节省时间和计算资源。
 - **泛化能力**：预训练模型捕获的通用语义知识可提升下游任务性能。
2. 自监督学习是一种通过构造数据内在监督信号进行训练的方法（例如通过掩码预测或句子顺序预测生成伪标签）。其优势包括：
- **无需人工标注**：利用海量无标注数据，突破标注瓶颈。
 - **通用表示学习**：通过设计合理的预训练任务，模型可学习到对下游任务有益的语义和语法知识。

3. 微调策略分析：

- **防止过拟合**：策略（1）（冻结预训练参数）更有效。少样本下，全量微调容易因参数过多而过拟合，而冻结参数仅训练输出头可保留预训练知识并减少可调参数量。
- **训练成本**：策略（2）（全量微调）成本更高。全量微调需计算所有参数的梯度并更新，而策略（1）仅需更新输出头参数。

4. 代码与运行结果

(a) 运行结果：

```
(pytorch) root@66308c6d2cf:~/RLP Homework II (2025 Spring)# ./root/miniconda3/envs/pytorch/bin/python "/root/RLP Homework II (2025 Spring)/demo_bert_ft.py"
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ./bert-mini-uncased/ and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Generating train split: 650000 examples [00:01, 639145.52 examples/s]
Generating test split: 50000 examples [00:00, 746646.92 examples/s]
Map: 100%
Map: 100%
Training Epoch 1, loss=1.5946; 100%
Evaluating: 100%
Validation accuracy: 0.2720
Training Epoch 2, loss=1.5229; 100%
Evaluating: 100%
Validation accuracy: 0.3080
Training Epoch 3, loss=1.3029; 100%
Evaluating: 100%
Validation accuracy: 0.3460
Training Epoch 4, loss=1.4820; 100%
Evaluating: 100%
Validation accuracy: 0.4240
Training Epoch 5, loss=1.4054; 100%
Evaluating: 100%
Validation accuracy: 0.4140
```

(b) 修改后代码：

```
1 import torch
2 from datasets import load_dataset
3 from torch.optim import AdamW
4 from torch.utils.data import DataLoader
5 from tqdm import tqdm
6 from transformers import BertTokenizer, BertForSequenceClassification
7
```

```
8 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
9
10 # SETTING MODEL
11 model_path = "./bert-mini-uncased/" # the path of BERT model
12 model = BertForSequenceClassification.from_pretrained(model_path, num_labels=5).to(
    device)
13
14 # 冻结预训练参数（核心修改部分） 冻结 BERT 主干的参数
15 for param in model.bert.parameters(): # 只冻结 BERT 编码器参数
16     param.requires_grad = False
17
18 tokenizer = BertTokenizer.from_pretrained(model_path)
19
20 # SETTING DATASETS
21 dataset_path = "./yelp_review_full/" # the path of YelpReview dataset
22 dataset = load_dataset(dataset_path)
23
24 small_train_dataset = dataset["train"].select(range(1000))
25 train_dataset = small_train_dataset.map(
26     lambda x: tokenizer(x["text"], padding="max_length", truncation=True),
27     batched=True,
28 )
29 train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
30 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
31
32 small_eval_dataset = dataset["test"].select(range(500))
33 eval_dataset = small_eval_dataset.map(
34     lambda x: tokenizer(x["text"], padding="max_length", truncation=True),
35     batched=True,
36 )
37 eval_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
38 eval_loader = DataLoader(eval_dataset, batch_size=16)
39
40 optimizer = AdamW(model.parameters(), lr=2e-5)
41
42 num_epochs = 5
43 for epoch in range(num_epochs):
44     # TRAIN
45     model.train()
46     train_tqdm_processor = tqdm(train_loader)
47     for batch in train_tqdm_processor:
48         optimizer.zero_grad()
49
50         input_ids = batch["input_ids"].to(device)
51         attention_mask = batch["attention_mask"].to(device)
52         label = batch["label"].to(device)
53
54         loss = model(input_ids, attention_mask=attention_mask, labels=label).loss
55         train_tqdm_processor.set_description(f"Training Epoch {epoch + 1}, loss={loss}.
```

```

item():.4f}"))
56
57     loss.backward()
58     optimizer.step()
59
60     # EVALUATION
61     model.eval()
62     correct = 0
63     total = 0
64     with torch.no_grad():
65         for batch in tqdm(eval_loader, desc="Evaluating"):
66             input_ids = batch["input_ids"].to(device)
67             attention_mask = batch["attention_mask"].to(device)
68             label = batch["label"].to(device)
69
70             outputs = model(input_ids, attention_mask=attention_mask)
71             predictions = torch.argmax(outputs.logits, dim=-1)
72
73             correct += (predictions == label).sum().item()
74             total += label.size(0)
75
76     accuracy = correct / total
77     print(f"Validation accuracy: {accuracy:.4f}")
78
79 # 验证冻结效果
80 total_params = sum(p.numel() for p in model.parameters())
81 trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
82 print(f"总参数: {total_params}, 可训练参数: {trainable_params} ({trainable_params/
83     total_params:.1%})")

```

Listing 1: 冻结预训练参数修改

运行结果:

```

(pytorch) root@66208c6d2cf:~/NLP Homework II (2025 Spring)# ./bert-mini-uncased/ ./bert-mini-uncased/2.py
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ./bert-mini-uncased/ and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Training epoch 1, loss=1.6389: 100% | 62/62 [00:01:00:00, 47.65it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 73.10it/s]
Validation accuracy: 0.2320
Training epoch 2, loss=1.5938: 100% | 62/62 [00:00:00:00, 63.75it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 77.38it/s]
Validation accuracy: 0.2420
Training epoch 3, loss=1.6015: 100% | 62/62 [00:00:00:00, 64.61it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 77.89it/s]
Validation accuracy: 0.2420
Training epoch 4, loss=1.5669: 100% | 62/62 [00:00:00:00, 64.60it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 78.13it/s]
Validation accuracy: 0.2500
Training epoch 5, loss=1.6077: 100% | 62/62 [00:00:00:00, 64.31it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 78.67it/s]
Validation accuracy: 0.2560
总参数: 11171845, 可训练参数: 1285 (0.0%)

```

(c) 修改后代码:

```

1 import torch
2 from datasets import load_dataset
3 from torch.optim import AdamW
4 from torch.utils.data import DataLoader

```

```
5 from tqdm import tqdm
6 from transformers import BertTokenizer, BertForSequenceClassification
7 from transformers import BertTokenizer, BertForSequenceClassification, BertConfig
8
9 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
10
11 # 创建新的模型配置
12 config = BertConfig.from_pretrained("./bert-mini-uncased/", num_labels=5)
13 model = BertForSequenceClassification(config).to(device) # 根据配置初始化新模型
14
15 tokenizer = BertTokenizer.from_pretrained("./bert-mini-uncased/")
16
17 # SETTING DATASETS
18 dataset_path = "./yelp_review_full/" # the path of YelpReview dataset
19 dataset = load_dataset(dataset_path)
20
21 small_train_dataset = dataset["train"].select(range(1000))
22 train_dataset = small_train_dataset.map(
23     lambda x: tokenizer(x["text"], padding="max_length", truncation=True),
24     batched=True,
25 )
26 train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
27 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True, drop_last=True)
28
29 small_eval_dataset = dataset["test"].select(range(500))
30 eval_dataset = small_eval_dataset.map(
31     lambda x: tokenizer(x["text"], padding="max_length", truncation=True),
32     batched=True,
33 )
34 eval_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
35 eval_loader = DataLoader(eval_dataset, batch_size=16)
36
37 optimizer = AdamW(model.parameters(), lr=2e-5)
38
39 num_epochs = 5
40 for epoch in range(num_epochs):
41     # TRAIN
42     model.train()
43     train_tqdm_processor = tqdm(train_loader)
44     for batch in train_tqdm_processor:
45         optimizer.zero_grad()
46
47         input_ids = batch["input_ids"].to(device)
48         attention_mask = batch["attention_mask"].to(device)
49         label = batch["label"].to(device)
50
51         loss = model(input_ids, attention_mask=attention_mask, labels=label).loss
52         train_tqdm_processor.set_description(f"Training Epoch {epoch + 1}, loss={loss.item():.4f}")
```

```

53
54     loss.backward()
55     optimizer.step()
56
57     # EVALUATION
58     model.eval()
59     correct = 0
60     total = 0
61     with torch.no_grad():
62         for batch in tqdm(eval_loader, desc="Evaluating"):
63             input_ids = batch["input_ids"].to(device)
64             attention_mask = batch["attention_mask"].to(device)
65             label = batch["label"].to(device)
66
67             outputs = model(input_ids, attention_mask=attention_mask)
68             predictions = torch.argmax(outputs.logits, dim=-1)
69
70             correct += (predictions == label).sum().item()
71             total += label.size(0)
72
73     accuracy = correct / total
74     print(f"Validation accuracy: {accuracy:.4f}")
75
76 print("参数示例 (应为随机值):")
77 for name, param in model.named_parameters():
78     if "classifier" in name:
79         print(f"{name}: \n{param.data[:2,:2]}\n")
80         break
81

```

Listing 2: 从零开始训练修改

运行结果:

```

Training Epoch 1, loss=1.6366: 100% | 62/62 [00:03:00:00, 17.161t/s]
Evaluating: 100% | 32/32 [00:00:00:00, 73.721t/s]
Validation accuracy: 0.2380
Training Epoch 2, loss=1.6542: 100% | 62/62 [00:02:00:00, 22.701t/s]
Evaluating: 100% | 32/32 [00:00:00:00, 72.171t/s]
Validation accuracy: 0.2380
Training Epoch 3, loss=1.6328: 100% | 62/62 [00:02:00:00, 22.511t/s]
Evaluating: 100% | 32/32 [00:00:00:00, 72.731t/s]
Validation accuracy: 0.2380
Training Epoch 4, loss=1.5416: 100% | 62/62 [00:02:00:00, 22.551t/s]
Evaluating: 100% | 32/32 [00:00:00:00, 70.391t/s]
Validation accuracy: 0.2380
Training Epoch 5, loss=1.6252: 100% | 62/62 [00:02:00:00, 21.531t/s]
Evaluating: 100% | 32/32 [00:00:00:00, 69.421t/s]
Validation accuracy: 0.2380
参数示例 (应为随机值):
classifier.weight:
tensor([[ -0.0016,  0.0362],
        [ 0.0245, -0.0187]], device='cuda:0')

```

III. 预训练语言模型 (24 points)

1. BERT 和 GPT 使用的预训练任务分别是什么? (3 points) BERT 和 GPT 在网络结构上的主要区别是什么? (2 points) BERT 和 GPT 分别适合什么样的下游任务? (2 points)
2. 基于 Transformers 架构的语言模型 (例如: BERT 和 GPT) 需要引入位置编码. 请从 Attention 机制的角度解释位置编码的必要性. (2 points) BERT 和 GPT 使用可训练的位置向量作为位置编码. 一般认为这种绝对位置编码的缺点是没有外推性, 请解释原因. (2 points) 另外一种常见的绝对位置编码形式, 即 Sinusoidal 位置编码:

$$\begin{cases} p_{k,2i} = \sin\left(k/10000^{2i/d}\right) \\ p_{k,2i+1} = \cos\left(k/10000^{2i/d}\right) \end{cases} \quad (4)$$

其中, $p_{k,2i}$ 和 $p_{k,2i+1}$ 表示位置 k 的编码向量的第 $2i$ 和 $2i+1$ 个分量; d 为位置向量的维度. 由于位置 $k_1 + k_2$ 的向量可以表示成位置 k_1 和位置 k_2 的向量组合, 所以 Sinusoidal 位置编码可能可以表达相对位置信息. 请推导这种位置向量组合的具体形式. (3 points)

[提示 3] $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$, $\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$.

3. 双向注意力和因果注意力在 BERT 和 GPT 中发挥着重要作用. 在 PyTorch 库中, 可以通过注意力接口函数 `torch.nn.functional.scaled_dot_product_attention()` 配合使用 Attention Mask 来实现对应的注意力机制. 双向注意力和因果注意力分别使用什么形式的 Attention Mask? (2 points)
4. 假设一个语言模型需要编码一句问答文本 [“What”, “is”, “your”, “name”, “?”, “My”, “name”, “is”, “Zhang”, “San”, “.”]. 这段问答文本一共有 10 个 token, 其中问题有 5 个 token, 回答有 5 个 token. 对应的 Attention Mask 是一个 10×10 的矩阵. 如果语言模型使用双向自注意力, 请写出 Attention Mask 矩阵. (2 points) 如果语言模型使用因果自注意力, 请写出 Attention Mask 矩阵. (2 points) 现在需要设计一种特殊的自注意力机制, 使得问题中的某个 token 只可以“注意”到之前的问题 token, 回答中的某个 token 可以“注意”到所有的问题 token 和所有的回答 token. 请写出这种自注意力机制的 Attention Mask 矩阵. (4 points)

解:

1. BERT 与 GPT 对比:

- 预训练任务:

- BERT: 掩码语言建模 (MLM) 和下一句预测 (NSP)
- GPT: 自回归语言建模 (预测下一个词)

- 网络结构区别:

- BERT: 使用 Transformer 编码器, 双向注意力机制
- GPT: 使用 Transformer 解码器, 带掩码的单向注意力

- **适用下游任务：**

- BERT：文本分类、问答、语义理解等需要全局上下文的任务
- GPT：文本生成、对话系统、续写等序列生成任务

2. 位置编码分析：

- **必要性：**Transformer 的自注意力机制本身不包含位置信息，需要通过位置编码显式注入词序信息，否则模型无法区分不同位置的相同词汇。
- **外推性问题：**可训练位置编码在训练时只见过有限长度（如 512）的位置索引，当推理时输入超过该长度，模型无法生成合理的位置表示。
- **Sinusoidal 位置编码推导：**设位置 k_1 和 k_2 的编码分别为 $\mathbf{p}_{k_1}$ 和 $\mathbf{p}_{k_2}$ ，其线性组合可表示为：

$$\mathbf{p}_{k_1+k_2} = \begin{bmatrix} \sin(\omega_i(k_1 + k_2)) \\ \cos(\omega_i(k_1 + k_2)) \end{bmatrix} = \begin{bmatrix} \sin(\omega_i k_1) \cos(\omega_i k_2) + \cos(\omega_i k_1) \sin(\omega_i k_2) \\ \cos(\omega_i k_1) \cos(\omega_i k_2) - \sin(\omega_i k_1) \sin(\omega_i k_2) \end{bmatrix}$$

其中 $\omega_i = 1/10000^{2i/d}$ 。可见 $\mathbf{p}_{k_1+k_2}$ 可由 $\mathbf{p}_{k_1}$ 和 $\mathbf{p}_{k_2}$ 的向量元素线性组合得到。

3. 注意力掩码类型：

- **双向注意力：**全 1 矩阵（无掩码），允许所有位置互相关注

$$\text{Mask}_{ij} = 1 \quad \forall i, j$$

- **因果注意力：**下三角矩阵（含对角线），仅允许关注当前位置及之前的 token

$$\text{Mask}_{ij} = \begin{cases} 1 & \text{if } j \leq i \\ 0 & \text{otherwise} \end{cases}$$

4. Attention Mask 设计：

- **双向自注意力：**

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}_{10 \times 10}$$

- **因果自注意力：**

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}_{10 \times 10}$$

- **特殊自注意力** (问题-回答分区):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

- 前 5 行 (问题部分) 使用因果注意力
- 后 5 行 (回答部分) 可访问全部问题 token 和因果访问回答 token

IV. 模型蒸馏 (24 points)

1. 什么是模型蒸馏? (2 points) 为什么需要模型蒸馏? (4 points)
2. 在 DistilBERT 中, 硬标签损失和软标签损失分别是什么? (2 points) 如果使用的数据集中无标签, 该如何设计蒸馏损失? (4 points)
3. 将题目 II. 中微调后的文本分类模型作为教师模型, 仅使用软标签损失作为蒸馏损失将其知识蒸馏到一个较小的模型 bert-tiny-uncased (该模型也包含在链接中) 中. 请提供关键代码和实验结果. (12 points)

解:

1. **模型蒸馏**是一种将复杂教师模型的知识迁移到轻量级学生模型的技术。其核心思想是通过教师模型的软标签 (soft labels) 指导学生模型的训练, 而不仅仅依赖原始数据标签。

必要性:

- **模型压缩:** 将参数量大的教师模型 (如 BERT-large) 压缩为更小的学生模型 (如 DistilBERT), 降低部署成本
- **推理加速:** 学生模型参数量减少 40% 以上, 推理速度提升 2-4 倍
- **知识迁移:** 通过软标签传递类别间相似性等暗知识 (dark knowledge), 提升学生模型的泛化能力
- **标签效率:** 在无标签数据场景中, 可利用教师模型生成伪标签进行训练

2. DistilBERT 的损失函数:

- **硬标签损失**：学生模型预测与真实标签的交叉熵损失

$$\mathcal{L}_{\text{hard}} = - \sum_{i=1}^N y_i \log p_i^{\text{student}}$$

- **软标签损失**：学生模型与教师模型输出的 KL 散度损失（含温度缩放）

$$\mathcal{L}_{\text{soft}} = T^2 \cdot \text{KL} \left(\frac{\mathbf{p}^{\text{teacher}}}{T} \parallel \frac{\mathbf{p}^{\text{student}}}{T} \right)$$

其中 T 为温度超参数（通常设为 2-5）

无标签数据蒸馏策略：

- **纯软标签损失**：仅使用教师模型的输出作为监督信号

$$\mathcal{L} = \mathcal{L}_{\text{soft}}$$

- **数据增强**：结合回译（back-translation）、随机掩码等增强方法生成伪数据
- **自蒸馏**：通过教师模型对无标签数据预测生成伪标签，迭代优化学生模型

3. 修改后代码：

```

1 import torch
2 from datasets import load_dataset
3 from torch.optim import AdamW
4 from torch.utils.data import DataLoader
5 from tqdm import tqdm
6 from transformers import BertTokenizer, BertForSequenceClassification
7 import torch.nn.functional as F
8
9 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
10
11 # 加载教师模型
12 teacher_path = "./bert-mini-uncased/"
13 teacher = BertForSequenceClassification.from_pretrained(teacher_path, num_labels=5).to(
14     device)
15 tokenizer = BertTokenizer.from_pretrained(teacher_path)
16
17 # 数据预处理
18 dataset = load_dataset("./yelp_review_full/")
19
20 # 创建小规模数据集
21 def preprocess(data):
22     return tokenizer(data["text"], padding="max_length", truncation=True, max_length=256)
23
24 train_data = dataset["train"].select(range(1000)).map(preprocess, batched=True)
25 eval_data = dataset["test"].select(range(500)).map(preprocess, batched=True)
26 train_data.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

```

```
26 eval_data.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
27
28 train_loader = DataLoader(train_data, batch_size=16, shuffle=True)
29 eval_loader = DataLoader(eval_data, batch_size=16)
30
31 print("===== 训练教师模型 =====")
32 optimizer = AdamW(teacher.parameters(), lr=2e-5)
33 for epoch in range(3): # 短训练演示
34     teacher.train()
35     for batch in tqdm(train_loader):
36         optimizer.zero_grad()
37         inputs = {k: v.to(device) for k, v in batch.items() if k != "label"}
38         loss = teacher(**inputs, labels=batch["label"].to(device)).loss
39         loss.backward()
40         optimizer.step()
41
42 # 初始化学生模型
43 student_path = "./bert-tiny-uncased/"
44 student = BertForSequenceClassification.from_pretrained(student_path, num_labels=5).to(
45     device)
46
47 # 蒸馏参数
48 T = 3 # 温度参数
49 optimizer = AdamW(student.parameters(), lr=5e-5)
50
51 print("\n===== 开始蒸馏训练 =====")
52 for epoch in range(5):
53     student.train()
54     teacher.eval()
55
56     total_loss = 0
57     for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}"):
58         optimizer.zero_grad()
59
60         # 移动数据到设备
61         inputs = {
62             "input_ids": batch["input_ids"].to(device),
63             "attention_mask": batch["attention_mask"].to(device)
64         }
65
66         # 生成教师软标签
67         with torch.no_grad():
68             teacher_logits = teacher(**inputs).logits
69             soft_targets = F.softmax(teacher_logits / T, dim=-1)
70
71         # 学生预测
72         student_logits = student(**inputs).logits
73         student_probs = F.log_softmax(student_logits / T, dim=-1)
```

```

74     # 计算KL散度损失
75     loss = F.kl_div(student_probs, soft_targets, reduction="batchmean") * (T**2)
76
77     loss.backward()
78     optimizer.step()
79     total_loss += loss.item()
80
81     # 评估
82     student.eval()
83     correct = 0
84     for batch in tqdm(eval_loader, desc="Evaluating"):
85         inputs = {
86             "input_ids": batch["input_ids"].to(device),
87             "attention_mask": batch["attention_mask"].to(device)
88         }
89         with torch.no_grad():
90             logits = student(**inputs).logits
91             preds = torch.argmax(logits, dim=1)
92             correct += (preds == batch["label"].to(device)).sum().item()
93
94     accuracy = correct / len(eval_data)
95     print(f"Epoch {epoch+1} | Loss: {total_loss/len(train_loader):.4f} | Acc: {accuracy:.4f}
96         ")

```

Listing 3: 蒸馏

运行结果：

```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ./bert-mini-uncased/ and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Map: 100% | 1000/1000 [00:01:00:00, 769.99 examples/s]
Map: 100% | 500/500 [00:00:00:00, 840.72 examples/s]
===== 训练数据模型 =====
100% | 63/63 [00:02:00:00, 28.74it/s]
100% | 63/63 [00:01:00:00, 44.05it/s]
100% | 63/63 [00:01:00:00, 44.32it/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ./bert-tiny-uncased/ and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
===== 开始蒸馏训练 =====
Epoch 1: 100% | 63/63 [00:00:00:00, 66.87it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 317.10it/s]
Epoch 1 | Loss: 0.0382 | Acc: 0.2380
Epoch 2: 100% | 63/63 [00:00:00:00, 73.53it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 372.69it/s]
Epoch 2 | Loss: 0.0219 | Acc: 0.2960
Epoch 3: 100% | 63/63 [00:00:00:00, 76.01it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 383.11it/s]
Epoch 3 | Loss: 0.0141 | Acc: 0.3160
Epoch 4: 100% | 63/63 [00:01:00:00, 40.62it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 424.75it/s]
Epoch 4 | Loss: 0.0107 | Acc: 0.3180
Epoch 5: 100% | 63/63 [00:00:00:00, 79.11it/s]
Evaluating: 100% | 32/32 [00:00:00:00, 425.35it/s]
Epoch 5 | Loss: 0.0084 | Acc: 0.3240

```